

tidyverse_naming_conventions

January 13, 2026 at 07:52 AM

Tidyverse Naming Conventions for R Objects (January 13, 2026)

General Principles

Use snake_case: Lowercase letters with underscores separating words. This applies to variables, functions, and arguments.

Be descriptive: Names should be meaningful and self-documenting. Prioritize clarity over brevity, but avoid excessive length.

Use consistent vocabulary: Choose one term and stick with it throughout a project (e.g., always `patient` or always `subject`, not both).

Variables

Use nouns that describe what the object contains.

Good	Bad	Why
<code>patient_age</code>	<code>patientAge</code>	Avoid camelCase
<code>first_visit_date</code>	<code>first.visit.date</code>	Dots reserved for S3 methods
<code>total_count</code>	<code>tc</code>	Cryptic abbreviations obscure meaning
<code>mean_bp</code>	<code>x</code>	Single letters lack context
<code>response_time_ms</code>	<code>data</code>	Avoid shadowing base R names
<code>is_treated</code>	<code>treated_flag</code>	Prefer <code>is_</code> prefix for booleans
<code>has_comorbidity</code>	<code>comorbidity_yn</code>	Avoid <code>_yn</code> , <code>_flag</code> suffixes

Data frames: Use plural nouns or collective terms.

`patients`
`lab_results`
`adverse_events`

Vectors/columns: Use singular nouns describing each element.

```
patient_id
systolic_bp
visit_date
```

Functions

Use verbs that describe the action performed. The name should indicate what the function does, not how it does it.

Good	Bad	Why
<code>calculate_bmi()</code>	<code>bmi()</code>	Verb clarifies it computes something
<code>import_lab_data()</code>	<code>getData()</code>	Avoid camelCase; be specific
<code>validate_dates()</code>	<code>check()</code>	Too vague
<code>summarize_by_group()</code>	<code>summ_grp()</code>	Excessive abbreviation
<code>fit_cox_model()</code>	<code>CoxModel()</code>	Functions should not look like classes
<code>extract_coefficients()</code>	<code>coef2()</code>	Avoid version-number suffixes

Predicate functions (returning TRUE/FALSE) should use `is_`, `has_`, `can_`:

```
is_valid_id()
has_missing_values()
can_merge_datasets()
```

Accessor functions can use `get_` or simply the noun:

```
get_patient_ids()
patient_ids()
```

Function Arguments

Use the same `snake_case` convention. Be explicit about what the argument controls.

Good	Bad	Why
<code>na_rm = TRUE</code>	<code>removeNA = TRUE</code>	Avoid camelCase
<code>include_baseline = FALSE</code>	<code>incl_bl = FALSE</code>	Avoid cryptic abbreviations
<code>confidence_level = 0.95</code>	<code>cl = 0.95</code>	Unclear meaning
<code>output_format = "csv"</code>	<code>fmt = "csv"</code>	Prefer full words
<code>max_iterations = 100</code>	<code>maxIter = 100</code>	Maintain snake_case

Names to Avoid

Do not shadow base R or common package functions:

```
c <- 10
mean <- "average"
data <- read_csv("file.csv")
df <- my_data_frame
T <- "treatment"
F <- 0
```

All of these will cause subtle bugs or confusion.

Avoid single letters except in limited contexts:

- i, j, k for loop indices
- n for counts
- x, y for coordinates or generic inputs in simple functions
- p for probabilities in statistical contexts

Avoid these patterns:

Pattern	Example	Problem
Dots	patient.data	Reserved for S3 dispatch
ALL_CAPS	MAX_VALUE	Convention for constants in other languages, not R
Leading dots	.hidden_var	Reserved for internal/hidden objects
Numbers as prefixes	1st_visit	Syntactically invalid
Special characters	patient@id	Syntactically invalid or confusing

Consistent Vocabulary

Within a project, choose one term for each concept:

Consistent	Inconsistent
Always patient_id	Mixing patient_id, pt_id, subject_id, id
Always visit_date	Mixing visit_date, date_of_visit, visitDate
Always calculate_*	Mixing calculate_*, compute_*, get_*

Summary

```
patient_demographics # data frame (plural noun)
enrollment_date      # variable (singular noun, snake_case)
is_randomized         # boolean (is_ prefix)
```

```
calculate_follow_up_time # function (verb, snake_case)
fit_survival_model       # function (verb describing action)
max_follow_up_days       # argument (descriptive, snake_case)
```